

```

#!/bin/bash
#
# run_all_tests.sh - automates running every glossary section's own unit
# test group through MAME, one at a time, capturing each group's output
# to its own log file.
#
# Requires: lwasm, MAME (mecb6809), python3. Assumes forth6809.asm and
# mame_listener.py / retrigger.cmd (this script's own siblings) are all
# reachable from the paths configured below - all overridable, run with
# --help to see every flag. retrigger.cmd itself is deliberately bare
# (MAME's debugger command language doesn't understand comments); see
# retrigger_notes.txt alongside it for what it does and why.
#
# How each iteration avoids the three problems raised:
#
# 1. Fixed serial connection: MAME is launched with
#    "-rs232 null_modem -bitb socket.HOST:PORT" instead of "-rs232 pty".
#    This is a fixed TCP endpoint every run - no device path to
#    discover via the MAME UI, ever.
#
# 2. Pausing until the listener can connect: mame_listener.py binds
#    and starts listening *before* MAME is launched. MAME's own
#    "-bitb socket.HOST:PORT" tries to CONNECT as a client first, so
#    it connects straight into the already-listening script as part
#    of its own startup - no timing race. "-debug" on top of that
#    means MAME also starts fully paused at the reset vector before
#    any code runs at all, confirmed directly from MAME's own docs.
#
# 3. Retriggering INITCODE programmatically: "-debugscript retrigger.cmd"
#    runs a one-line "go" debugger command automatically the moment
#    MAME starts, resuming from the already-paused reset vector - no
#    manual typing into a debugger window required.
#
# 4. Collecting all test results: mame_listener.py logs every byte
#    MAME sends down that same TCP socket straight to a file, using
#    an idle timeout (silence means TSTRUNNER has returned) backed by
#    a hard timeout (guards against a genuine hang, the same class of
#    bug this project has hit before with BASE=0).
#
# NONE of this has been run against a real mecb6809 build - it rests on
# MAME's own documented behavior for -debug/-debugscript/-bitb, not on
# an actual test. Confirm "-rs232 null_modem" and "-bitb socket.host:port"
# are genuinely supported by this driver first (see the sanity-check
# block below) before trusting a long unattended run.

```

```
set -uo pipefail
```

```
# -----
# Configuration - built-in defaults, all overridable from the command
# line below (run with --help to see every flag)
# -----
ASM_SOURCE="forth6809.asm"
LWASM_BIN="lwasm"
MAME_BIN="./mecb6809"
MAME_SYSTEM="mecb6809"
ROM_DEST="$HOME/Library/Application Support/mame/roms/mecb6809/mecb6809.bin"
LISTENER_PORT=2000
LISTENER_HOST=127.0.0.1
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
RETRIGGER_CMD="$SCRIPT_DIR/retrigger.cmd"
LISTENER_PY="$SCRIPT_DIR/mame_listener.py"
LOG_DIR="$SCRIPT_DIR/results"
IDLE_TIMEOUT=5
HARD_TIMEOUT=60
```

```
usage() {
    cat <<EOF
Usage: $0 [OPTIONS]
```

All options have built-in defaults (shown below); only pass what you need to override for your own machine.

--asm-source PATH	Path to forth6809.asm (default: \$ASM_SOURCE)
--lwasm-bin PATH	lwasm executable (default: \$LWASM_BIN)
--mame-bin PATH	MAME executable (default: \$MAME_BIN)
--mame-system NAME	MAME system/driver name (default: \$MAME_SYSTEM)
--rom-dest PATH	Where to copy the built ROM (default: \$ROM_DEST)
--listener-port PORT	TCP port for the capture listener (default: \$LISTENER_PO
--listener-host HOST	Host/IP for the capture listener (default: \$LISTENER_HOS
--idle-timeout SECS	Seconds of silence = test group done (default: \$IDLE_TIM
--hard-timeout SECS	Absolute max seconds per test group (default: \$HARD_TIME
-h, --help	Show this help and exit

```
EOF
}
```

```
while [[ $# -gt 0 ]]; do
    case "$1" in
        --asm-source)    ASM_SOURCE="$2"; shift 2 ;;
        --lwasm-bin)     LWASM_BIN="$2"; shift 2 ;;
        --mame-bin)      MAME_BIN="$2"; shift 2 ;;
        --mame-system)   MAME_SYSTEM="$2"; shift 2 ;;
```

```

--rom-dest)      ROM_DEST="$2"; shift 2 ;;
--listener-port) LISTENER_PORT="$2"; shift 2 ;;
--listener-host) LISTENER_HOST="$2"; shift 2 ;;
--idle-timeout)  IDLE_TIMEOUT="$2"; shift 2 ;;
--hard-timeout)  HARD_TIMEOUT="$2"; shift 2 ;;
-h|--help)       usage; exit 0 ;;
*)               echo "Unknown option: $1" >&2; usage; exit 1 ;;
esac
done

# Glossary section names, in TSTSELECTOR order (0-17), matching the
# table in the ClaudeForth documentation's own Build Instructions
# section - used only for readable log file names and the summary.
SECTION_NAMES=(
    "3.1_SysIO" "3.2_Stack" "3.3_RetStack" "3.4_SArith" "3.5_DArith"
    "3.6_Logic" "3.7_Compare" "3.8_CtrlFlow" "3.9_DefWords" "3.10_CompWords"
    "3.11_Memory" "3.12_StrParse" "3.13_NumOut" "3.14_BaseRadix"
    "3.15_Exception" "3.16_Comments" "3.17_EnvSys" "3.18_Tools"
)

mkdir -p "$LOG_DIR"

# -----
# One-time sanity check: confirm this driver actually exposes the
# rs232/null_modem/bitb combination before looping 18 times on a
# machine config that might not support it.
# -----
echo "=== Sanity check: does $MAME_SYSTEM support -rs232 null_modem -bitb? ==="
"$MAME_BIN" "$MAME_SYSTEM" -listslots 2>&1 | grep -i rs232 || {
    echo "WARNING: no 'rs232' slot found in -listslots output."
    echo "This automation assumes the same rs232 slot the documented"
    echo "'-rs232 pty' command already uses - if that warning is genuine,"
    echo "stop here and check '$MAME_BIN $MAME_SYSTEM -listmedia' by hand"
    echo "before trusting the rest of this script."
}
echo

# -----
# Main loop
# -----
declare -a RESULTS

for ((n=0; n<18; n++)); do
    name="{SECTION_NAMES[$n]}"
    logfile="$LOG_DIR/${n}_${name}.log"

    echo "======"

```

```

echo "=== TSTSELECTOR=$n ($name) ==="
echo "=====

# Step 1: assemble this specific test group into the ROM image
"$LWASM_BIN" --6809 --format=raw --output=forth6809.bin \
  --list=forth6809.lst \
  --define=UNITTESTS=1 --define=TSTSELECTOR="$n" \
  "$ASM_SOURCE"
if [ $? -ne 0 ]; then
  echo "FAIL: lwasm failed for TSTSELECTOR=$n"
  RESULTS+=("$n $name ASSEMBLE_FAIL")
  continue
fi

# Step 2: install it where MAME will find it
cp forth6809.bin "$ROM_DEST"

# Step 3: start the listener first, so MAME connects straight into it.
# Its own stderr (progress/diagnostic messages, including the "hard
# timeout" marker checked for below) is captured separately from the
# logfile, which holds only the raw bytes MAME itself sent.
listener_stderr="$LOG_DIR/${n}_${name}.listener_stderr.log"
python3 "$LISTENER_PY" "$LISTENER_PORT" "$logfile" \
  --idle-timeout "$IDLE_TIMEOUT" --hard-timeout "$HARD_TIMEOUT" \
  --host "$LISTENER_HOST" 2>"$listener_stderr" &
listener_pid=$!

# Give the listener a brief moment to actually reach listen() before
# MAME tries to connect - this is generous, not a fragile race: even
# if MAME connects a few hundred ms sooner, MAME's own client-connect
# retry behavior and TCP's own backlog handling make this robust in
# practice, but the sleep costs nothing and removes any doubt.
sleep 0.3

# Step 4: launch MAME, paused at reset, socket already connecting,
# debugscript will "go" the instant startup completes
"$MAME_BIN" "$MAME_SYSTEM" \
  -rs232 null_modem -bitb "socket.${LISTENER_HOST}:${LISTENER_PORT}" \
  -debug -debugscript "$RETRIGGER_CMD" \
  -window -resolution 640x480 \
  >"$LOG_DIR/${n}_${name}.mame_stdout.log" 2>&1 &
mame_pid=$!

# Step 5: wait for the listener to decide the run is done (idle or
# hard timeout, or MAME closing the connection on its own)
wait "$listener_pid"
listener_exit=$?

```

```

# Step 6: MAME may still be sitting there after the listener gives up
# (e.g. hard timeout case) - always terminate it explicitly rather
# than assuming it already exited.
if kill -0 "$mame_pid" 2>/dev/null; then
    kill "$mame_pid" 2>/dev/null
    sleep 0.5
    kill -9 "$mame_pid" 2>/dev/null
fi
wait "$mame_pid" 2>/dev/null

# Step 7: classify the result from the captured log
status="UNKNOWN"
if [ "$listener_exit" -eq 2 ]; then
    status="NO_CONNECTION"
elif grep -q "hard timeout" "$listener_stderr" 2>/dev/null; then
    status="HANG_SUSPECTED"
elif grep -qi "FAIL" "$logfile" 2>/dev/null; then
    status="TEST_FAILURE"
elif [ -s "$logfile" ]; then
    status="PASS"
else
    status="EMPTY_LOG"
fi

echo "Result: $status (log: $logfile)"
RESULTS+=("$n $name $status")
echo
done

# -----
# Summary
# -----
echo "======"
echo "=== SUMMARY ==="
echo "======"
printf "%-4s %-16s %s\n" "N" "SECTION" "STATUS"
for r in "${RESULTS[@]}; do
    # deliberately unquoted: each stored result is "N NAME STATUS" and word-
    # splitting it here is what feeds the 3 separate %s fields below; safe
    # since SECTION_NAMES above never contains spaces
    printf "%-4s %-16s %s\n" $r
done

```